

Laporan Tugas Kecil 3
IF2211 - Strategi Algoritma
Penyelesaian *Ice Sliding Puzzle* Menggunakan Algoritma
Pathfinding* UCS, GBFS, dan A



Disusun oleh:

Wisa Ahmaduta Dinutama/18223003

Muhammad Adam Mirza/18223015

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

I. Pendahuluan

Ice Sliding Puzzle adalah permainan logika di mana pemain menggerakkan karakter (Z) dari titik awal menuju titik tujuan (O) di atas permukaan es yang licin. Karakter bergerak secara horizontal atau vertikal dan tidak akan berhenti sampai menabrak dinding/rintangan (X). Terdapat modifikasi berupa checkpoint bernomor (0, 1, 2, ...9) yang harus dilalui secara berurutan, tile Lava (L) yang menyebabkan game over, serta cost traversal pada setiap tile. Tugas kecil ini mencakup pembuatan program untuk melakukan pencarian jalur guna menyelesaikan *puzzle* yang bersangkutan dengan mengimplementasikan tiga algoritma *pathfinding*, yaitu UCS, Greedy Best-First Search (GBFS), dan A-Star (A*).

II. Algoritma Pathfinding yang Digunakan

A. Komponen Umum

Sebelum masing-masing algoritma *pathfinding* yang kami implementasikan dijelaskan satu per satu, ada baiknya kami menjelaskan terlebih dahulu beberapa komponen umum yang digunakan secara berulang oleh ketiga algoritma untuk penjelasan yang lebih modular. Perbedaan antara algoritma hanya terletak pada fungsi prioritas dan strategi closed list, yang akan dijelaskan pada masing-masing bagian khusus untuk tiap algoritma. Adapun komponen umum yang kami definisikan adalah sebagai berikut.

1. Representasi State

Setiap state merepresentasikan kondisi permainan pada suatu saat, terdiri dari:

- a) X & Y: Posisi pemain pada papan.
- b) NextNumber: nomor checkpoint <i>berikutnya</i> yang harus dilalui
- c) Cost: total biaya akumulatif ($g(n)$) dari state awal hingga state saat ini
- d) Parent: pointer ke state sebelumnya untuk rekonstruksi jalur solusi.

Selain itu, kami juga mendefinisikan StateKey sebagai key untuk map yang akan digunakan untuk deteksi state duplikat terdiri dari (X, Y, NextNumber). Hal ini karena dua state pada posisi yang sama bisa memiliki progress checkpoint <i>berbeda</i>

2. Mekanisme Generasi State Berikutnya

Untuk setiap state, program mencoba menggerakkan pemain ke empat arah: UP, DOWN,

LEFT, RIGHT. Setiap gerakan tersebut akan menjadi kemungkinan state berikutnya (maksimal 4). Setiap gerakan disimulasikan dengan langkah-langkah berikut:

- a) Aktor bergerak terus-menerus satu tile per iterasi ke arah yang dipilih.
- b) Setiap tile yang dilalui ditambahkan cost traversal-nya ke cost akumulatif.
- c) Jika tile berikutnya adalah dinding (X), aktor berhenti tepat sebelum dinding tersebut.
- d) Jika tile berikutnya berada di luar papan atau merupakan Lava (L), gerakan ini dianggap tidak valid (game over) dan state baru tidak dihasilkan.
- e) Jika aktor melewati <i>, diperiksa apakah nomor tersebut sama dengan NextNumber:
 - (1) Jika sesuai urutan, NextNumber di-increment.
 - (2) Jika tidak sesuai, gerakan dianggap tidak valid.
- f) Jika posisi akhir sama dengan posisi awal, gerakan tidak valid.

3. Kondisi Goal

Sebuah state dikatakan solusi jika memenuhi dua syarat secara bersamaan:

- a) Posisi aktor berada tepat di tile tujuan (O)
- b) Semua checkpoint bernomor telah dilalui secara berurutan

4. Proses Umum Algoritma Pencarian

Ketiga algoritma mengikuti kerangka pencarian berbasis priority queue yang sama:

a) Inisialisasi

Buat Priority Queue sebagai open list. Masukkan state awal ke dalam priority queue dengan prioritas sesuai fungsi evaluasi algoritma. Buat closed list untuk melacak state yang sudah dikunjungi.

b) Ambil State Prioritas Tertinggi

Ambil (pop) elemen teratas, yaitu state dengan nilai fungsi evaluasi terkecil.

c) Cek Duplikasi dan Update Closed List

Buat key dari state saat ini. Periksa apakah key tersebut sudah ada di closed list:

- (1) Jika sudah ada dan tidak lebih baik, lewati state ini dan kembali ke Langkah b
- (2) Jika belum ada atau lebih baik, update closed list. Lanjutkan ke Langkah d.

Perlu diingat bahwa definisi lebih baik berbeda untuk tiap algoritma

d) Cek Kondisi Goal

Periksa kondisi goal. Jika terpenuhi, solusi ditemukan. Lakukan rekonstruksi jalur melalui pointer Parent. Jika tidak, lanjut ke Langkah e.

e) Ekspansi

Generasi kandidat state berikutnya. Setiap state baru yang valid dimasukkan ke priority queue dengan prioritas sesuai fungsi evaluasi algoritma.

f) Ulangi kembali langkah b hingga solusi ditemukan atau priority queue kosong (tidak ada solusi)

B. Uniform Cost Search (UCS)

UCS disebut juga sebagai ekuivalen dari algoritma Dijkstra yang dibuat dengan fungsi yang lebih spesifik, yaitu mencari jarak terpendek antara dua simpul. UCS termasuk ke dalam algoritma *pathfinding* yang menjamin ditemukannya solusi dengan cost kumulatif minimum.

Adapun hal yang membedakan UCS dari kerangka umum:

1. Fungsi Evaluasi:

$$f(n) = g(n)$$

Prioritas ditentukan semata-mata oleh cost kumulatif aktual dari state awal ke state n.

Tidak ada komponen heuristik.

2. Strategi Closed List

Closed list menyimpan StateKey beserta cost g terbaik yang pernah ditemukan. Sebuah state dilewati jika cost lama kurang dari sama dengan cost saat ini. Jika cost saat ini lebih kecil, closed list diperbarui dan state diekspansi ulang.

3. Optimalitas:

Dijamin optimal karena state dengan cost terkecil selalu diekspansi terlebih dahulu. State pertama yang mencapai goal pasti memiliki cost minimum.

C. Greedy Best-First Search (GBFS)

GBFS adalah algoritma pencarian yang mengekskansi state berdasarkan estimasi jarak ke

goal (heuristik) saja, tanpa mempertimbangkan cost yang telah ditempuh.

Adapun hal yang membedakan GBFS dari kerangka umum:

1. Fungsi Evaluasi:

$$f(n) = h(n)$$

Prioritas ditentukan semata-mata oleh nilai heuristik, yaitu estimasi cost dari state n menuju goal. Cost kumulatif $g(n)$ diabaikan sepenuhnya.

Heuristik	yang	Digunakan:
$h(n) = \text{Total Manhattan Distance Berjenjang (Sequential Manhattan)}$		

Berbeda dengan jarak Manhattan standar, heuristik ini memperhitungkan kewajiban aktor untuk mengambil angka secara berurutan. Estimasi biaya dihitung dengan menjumlahkan jarak Manhattan dari posisi aktor saat ini (x_n, y_n) ke angka target berikutnya (m) , kemudian ditambah dengan akumulasi jarak antar angka yang tersisa hingga mencapai titik tujuan akhir (O) :

$$h(n) = |x_n - x_m| + |y_n - y_m| + \sum_{i=m}^K (|x_i - x_{i+1}| + |y_i - y_{i+1}|)$$

2. Strategi Closed List:

Closed list berupa set of StateKey. Sebuah state dilewati jika sudah pernah dikunjungi, tanpa membandingkan cost.

3. Optimalitas:

Tidak dijamin optimal. GBFS bersifat greedy, yaitu memilih jalur yang tampak paling dekat ke tujuan tanpa mempertimbangkan biaya yang telah ditempuh. Solusi yang ditemukan bisa memiliki cost jauh lebih besar dari minimum.

D. A-Star (A*)

A* menggabungkan optimalitas UCS dan efisiensi GBFS dengan menggunakan penjumlahan cost kumulatif dan heuristik sebagai fungsi evaluasi.

Adapun hal yang membedakan A* dari kerangka umum:

1. Fungsi Evaluasi:

$$f(n) = g(n) + h(n)$$

Prioritas ditentukan oleh penjumlahan cost kumulatif aktual $g(n)$ dan estimasi heuristik $h(n)$. Ini menyeimbangkan eksplorasi jalur murah dengan arah menuju goal.

2. Heuristik yang Digunakan:

Heuristik	yang	Digunakan:
$h(n)$	Total Manhattan Distance Berjenjang (Sequential Manhattan)	(Sama dengan yang digunakan pada GBFS). Heuristik dihitung dengan menjumlahkan jarak dari posisi saat ini ke target angka berikutnya, ditambah akumulasi jarak antar target angka yang tersisa hingga titik keluar (O):

$$h(n) = |x_n - x_m| + |y_n - y_m| + \sum_{i=m}^K (|x_i - x_{i+1}| + |y_i - y_{i+1}|)$$

3. Strategi Closed List:

Sama dengan UCS, closed list menyimpan StateKey beserta cost g terbaik. State dilewati jika cost g lama lebih kecil sama dengan cost g saat ini, dan akan diperbarui jika ditemukan cost g yang lebih kecil.

4. Optimalitas:

Dijamin optimal JIKA heuristik bersifat admissible (tidak pernah overestimate cost sebenarnya). Adapun analisis *admissibility* dibahas di BAB berikutnya.

III. Analisis Algoritma Pathfinding

A. Definisi $f(n)$ dan $g(n)$ pada Algoritma

1. $g(n)$:

$g(n)$ adalah cost kumulatif aktual dari state awal (posisi pemain mula-mula) menuju state n . Cost ini dihitung dari penjumlahan kumulatif cost traversal setiap tile yang dilalui aktor selama melakukan gerakan sliding. Tile posisi awal gerakan tidak dihitung, tetapi tile tempat pemain berhenti tetap dihitung. Sebagai contoh, jika aktor melakukan tiga gerakan dan melewati tile-tile dengan cost $\{3\}$, $\{2, 8\}$, dan $\{5, 4\}$, maka $g(n) = (3) + (2+8) + (5+4) = 22$.

2. $f(n)$:

$f(n)$ adalah fungsi evaluasi total yang digunakan untuk menentukan prioritas ekspansi suatu state. Definisi $f(n)$ berbeda untuk setiap algoritma:

a) UCS: $f(n) = g(n)$

Prioritas hanya berdasarkan cost aktual. Tidak ada komponen heuristik.

b) GBFS: $f(n) = h(n)$

Prioritas hanya berdasarkan estimasi heuristik. Cost aktual diabaikan sepenuhnya.

c) A*: $f(n) = g(n) + h(n)$

Prioritas berdasarkan penjumlahan cost aktual dan estimasi heuristik. Ini memberikan keseimbangan antara eksplorasi jalur murah (UCS) dan arah menuju goal (GBFS).

3. Definisi $h(n)$:

Definisi $h(n)$: $h(n)$ adalah fungsi heuristik yang mengestimasi cost dari state n menuju goal. Dalam implementasi ini, heuristik utama yang digunakan adalah *Sequential Manhattan Distance* (Jarak Manhattan Berjenjang):

$$h(n) = |x_n - x_m| + |y_n - y_m| + \sum_{i=m}^K (|x_i - x_{i+1}| + |y_i - y_{i+1}|)$$

di mana (x_n, y_n) adalah posisi aktor pada state n , (x_m, y_m) adalah posisi angka target berikutnya yang harus diambil, dan T_{K+1} adalah posisi tujuan akhir (O).

Heuristik ini tidak hanya mengestimasi jarak posisi ke titik keluar saja, tetapi secara eksplisit memperhitungkan constraint checkpoint bernomor. Dengan menghitung akumulasi jarak Manhattan yang harus ditempuh melalui setiap angka yang tersisa secara berurutan, fungsi heuristik memberikan bimbingan yang sangat kuat bagi algoritma untuk mengikuti jalur yang valid. Hal ini meningkatkan efisiensi pencarian secara signifikan karena state yang menjauhi target angka berikutnya akan memiliki nilai $h(n)$ yang sangat besar.

B. Apakah Heuristik yang Digunakan pada A* Admissible?

Sebuah heuristik $h(n)$ dikatakan *admissible* jika untuk setiap *state* n , nilai estimasi tidak pernah melebihi biaya sebenarnya menuju *goal* ($h(n) \leq h^*(n)$). Dalam implementasi *Ice Sliding Puzzle Solver* ini, heuristik *Sequential Manhattan Distance* terbukti bersifat *admissible* berdasarkan pertimbangan berikut:

Estimasi Batas Bawah (Lower Bound): Jarak Manhattan mengasumsikan perpindahan antar ubin dengan biaya minimum sebesar 1 per unit jarak. Karena setiap ubin pada papan permainan memiliki nilai *cost traversal* ≥ 1 (bilangan positif), maka biaya aktual ($h^*(n)$) untuk berpindah antar titik pasti akan selalu lebih besar atau sama dengan jarak Manhattan-nya.

Kepatuhan terhadap Checkpoint: Aturan permainan mewajibkan pengumpulan angka secara berurutan $(0, 1, \dots, K)$. Heuristik ini menghitung total jarak minimum yang harus ditempuh untuk mengunjungi seluruh urutan angka tersebut hingga ke tujuan akhir. Karena jalur optimal sebenarnya ($h^*(n)$) wajib melalui titik-titik tersebut, dan heuristik kita mengestimasi setiap segmen perjalanan menggunakan jarak terpendek (Manhattan), maka total estimasi $h(n)$ tidak akan pernah melebihi biaya jalur yang sebenarnya diambil oleh aktor.

Kendala Pergerakan (Sliding Mechanics): Mekanisme *sliding* mengakibatkan aktor tidak bisa berhenti di sembarang ubin, melainkan harus menabrak rintangan. Hal ini seringkali

memaksa aktor untuk mengambil rute memutar agar dapat mendarat tepat di atas ubin angka atau titik tujuan. Adanya rintangan (X) dan Lava (L) juga semakin memperbesar biaya aktual dibanding estimasi garis lurus Manhattan.

Kesimpulan Admissibility: Karena $h(n)$ selalu menghasilkan estimasi biaya yang lebih kecil atau sama dengan biaya optimal sebenarnya ($h(n) \leq h^*(n)$), maka heuristik ini *admissible*. Hal ini menjamin bahwa algoritma A akan selalu menemukan solusi dengan total *cost* paling minimum.

C. Perbandingan Algoritma Secara Teoritis

Aspek	UCS	GBFS	A*
Fungsi Evaluasi	$f(n) = g(n)$	$f(n) = h(n)$	$f(n) = g(n) + h(n)$
Optimalitas	Ya (dijamin optimal)	Tidak (suboptimal)	Ya (jika h admissible)
Kelengkapan	Ya (jika $cost > 0$)	Ya (pada graf berhingga)	Ya
Kompleksitas Waktu dan Ruang	$O(b^{(C^*/\epsilon)})$	$O(b^m)$	$O(b^{(C^*/\epsilon)})$
Penggunaan Ideal	Butuh optimalitas mutlak	Butuh eksekusi tercepat	Butuh optimal & efisien

Analisis teoritis:

1. UCS

Akan mengeksplorasi banyak state secara merata ke segala arah karena tidak memiliki informasi arah menuju goal. Pada papan yang besar atau kompleks, jumlah iterasi bisa sangat tinggi. Namun, solusi yang ditemukan dijamin memiliki cost minimum.

2. GBFS

Cenderung langsung menuju arah goal sehingga jumlah node yang diekspansi biasanya paling sedikit. Namun, pada Ice Sliding Puzzle, GBFS sering menemui masalah karena:

- a) Mekanisme sliding membuat aktor tidak selalu bisa bergerak langsung ke arah goal.
- b) Adanya checkpoint bernomor $\langle i \rangle$ yang harus dilalui berurutan bisa membuat arah optimal justru menjauhi goal terlebih dahulu.
- c) Solusi yang ditemukan tidak dijamin optimal (cost bisa jauh lebih besar dari minimum).

3. A*

Memberikan keseimbangan terbaik. Komponen $g(n)$ memastikan algoritma tidak mengabaikan cost yang telah ditempuh, sementara $h(n)$ memberikan arah pencarian menuju goal. Pada permasalahan ini, A* diharapkan:

- a) Menemukan solusi optimal (sama seperti UCS).
- b) Mengekspansi lebih sedikit node dibandingkan UCS karena heuristik membantu mengarahkan pencarian.
- c) Mengekspansi lebih banyak node dibandingkan GBFS, tetapi dengan jaminan optimalitas.

IV. Analisis Algoritma Pathfinding

Program ditulis dalam Golang. Berikut adalah source code dari setiap komponen program.

Struktur Direktori:

```
.
├── src/
│   ├── go.mod
│   ├── main.go
│   ├── static/
│   │   ├── index.html
│   │   └── app.js
│   ├── model/
│   │   ├── direction.go
│   │   ├── state.go
│   │   ├── priorityQueue.go
│   │   ├── solver.go
│   │   ├── algorithms.go
│   │   └── heuristic.go
│   └── utils/
│       └── parser.go
├── test/
├── docs/
└── README.md
```

Komponen	Kode
parser.go	<pre>package utils import ("IceSlidingPuzzle/model" "bufio" "fmt" "io"</pre>

```

        "strconv"
        "strings"
    )

    func ParseReader(r io.Reader) (model.State, model.Solver) {
        scanner := bufio.NewScanner(r)
        scanner.Scan()

        nm := strings.Fields(scanner.Text())
        n, _ := strconv.Atoi(nm[0])
        m, _ := strconv.Atoi(nm[1])

        var initial model.State
        var solver model.Solver

        board := make([][]byte, n)
        numberSequence := [10]bool{}
        for i := range n {
            scanner.Scan()
            line := scanner.Text()
            if len(line) != m {
                panic("invalid board input width")
            }
            board[i] = make([]byte, m)
            for j := range m {
                c := line[j]
                if c >= '0' && c <= '9' {
                    if numberSequence[c-'0'] {
                        panic("overlapping <i> detected")
                    }
                    numberSequence[c-'0'] = true
                } else if c != '*' && c != 'X' && c != 'L' && c != 'Z'
&& c != 'O' {
                    panic(fmt.Sprintf("%c is not a valid tile", c))
                }
                switch c {
                case 'Z':
                    initial.X = j
                    initial.Y = i
                    board[i][j] = '*'
                case 'O':

```

```

        solver.GoalX = j
        solver.GoalY = i
        board[i][j] = 'O'
    default:
        board[i][j] = c
    }
}

changed := false
prevNumber := numberSequence[0]
if prevNumber {
    initial.NextNumber = 0
}
for i := 1; i < len(numberSequence); i++ {
    if numberSequence[i] != prevNumber {
        if !changed {
            if !prevNumber {
                panic("given <i> is not sequential")
            }
            changed = true
            solver.FinalNumber = i-1
        } else if changed {
            panic("given <i> is not sequential")
        }
    }
    prevNumber = numberSequence[i]
}

costs := make([][]int, n)
for i := range n {
    scanner.Scan()
    fields := strings.Fields(scanner.Text())
    if len(fields) != m {
        panic("invalid costs input width")
    }
    costs[i] = make([]int, m)
    for j := range m {
        integer, _ := strconv.Atoi(fields[j])
        costs[i][j] = integer
    }
}

```

	<pre> solver.Board = board solver.Costs = costs return initial, solver } </pre>
Algorithms.g 0	<pre> package model import ("container/heap") type AlgorithmResult struct { FinalState State Solved bool Iterations int } type HeuristicFunc func(st State, s Solver) int func (s Solver) UCS(st State) AlgorithmResult { return s.searchWithPriorityQueue(st, func(state State) int { return state.Cost }) } func (s Solver) GBFS(st State, h HeuristicFunc) AlgorithmResult { return s.searchWithPriorityQueue(st, func(state State) int { return h(state, s) }) } func (s Solver) AStar(st State, h HeuristicFunc) AlgorithmResult { return s.searchWithPriorityQueue(st, func(state State) int { return state.Cost + h(state, s) }) } func (s Solver) searchWithPriorityQueue(st State, calculatePriority func(State) int) AlgorithmResult { open := &PriorityQueue{} heap.Init(open) </pre>

```

        heap.Push(open, &Item{
            State:    &st,
            Priority: calculatePriority(st),
        })

    close := make(map[StateKey]int)
    iterations := 0

    for open.Len() > 0 {
        iterations++
        item := heap.Pop(open).(*Item)
        cur := item.State

        key := cur.Key()

        if oldCost, ok := close[key]; ok && oldCost <= cur.Cost {
            continue
        }
        close[key] = cur.Cost

        if s.isSolved(*cur) {
            return AlgorithmResult{FinalState: *cur, Solved: true,
Iterations: iterations}
        }

        nextStates := s.generateNextmoves(*cur)

        for i := range nextStates {
            ns := nextStates[i]
            heap.Push(open, &Item{
                State:    &ns,
                Priority: calculatePriority(ns),
            })
        }
    }

    return AlgorithmResult{Solved: false, Iterations: iterations}
}

func (s Solver) BFS(st State) AlgorithmResult {

```

	<pre> queue := []State{st} visited := make(map[StateKey]bool) iterations := 0 for len(queue) > 0 { iterations++ cur := queue[0] queue = queue[1:] key := cur.Key() if visited[key] { continue } visited[key] = true if s.isSolved(cur) { return AlgorithmResult{FinalState: cur, Solved: true, Iterations: iterations} } nextStates := s.generateNextmoves(cur) for _, ns := range nextStates { if !visited[ns.Key()] { queue = append(queue, ns) } } } return AlgorithmResult{Solved: false, Iterations: iterations} } </pre>
direction.go	<pre> package model type Direction int const (UP Direction = iota DOWN RIGHT LEFT) </pre>

heuristic.go

```
package model

import "math"

func FindTargets(s Solver) map[int][2]int {
    targets := make(map[int][2]int)
    for y := 0; y < len(s.Board); y++ {
        for x := 0; x < len(s.Board[0]); x++ {
            c := s.Board[y][x]
            if c >= '0' && c <= '9' {
                targets[int(c-'0')] = [2]int{x, y}
            }
        }
    }
    targets[s.FinalNumber+1] = [2]int{s.GoalX, s.GoalY}
    return targets
}

func H1Manhattan(targets map[int][2]int) HeuristicFunc {
    return func(st State, s Solver) int {
        return calculateDistance(st, s, targets, func(dx, dy float64)
float64 {
            return math.Abs(dx) + math.Abs(dy)
        })
    }
}

func H2Euclidean(targets map[int][2]int) HeuristicFunc {
    return func(st State, s Solver) int {
        return calculateDistance(st, s, targets, func(dx, dy float64)
float64 {
            return math.Sqrt(dx*dx + dy*dy)
        })
    }
}

func H3Chebyshev(targets map[int][2]int) HeuristicFunc {
    return func(st State, s Solver) int {
        return calculateDistance(st, s, targets, func(dx, dy float64)
float64 {
            return math.Max(math.Abs(dx), math.Abs(dy))
        })
    }
}
```

	<pre> }) } } func H4MissingTargets() HeuristicFunc { return func(st State, s Solver) int { return (s.FinalNumber + 1 - st.NextNumber) } } func H5StraightToGoal(targets map[int][2]int) HeuristicFunc { return func(st State, s Solver) int { goalTarget := targets[s.FinalNumber+1] dx := math.Abs(float64(st.X - goalTarget[0])) dy := math.Abs(float64(st.Y - goalTarget[1])) return int(dx + dy) } } func calculateDistance(st State, s Solver, targets map[int][2]int, distFunc func(dx, dy float64) float64) int { cost := 0.0 currX, currY := st.X, st.Y for i := st.NextNumber; i <= s.FinalNumber+1; i++ { target, exists := targets[i] if !exists { continue } dx := float64(currX - target[0]) dy := float64(currY - target[1]) cost += distFunc(dx, dy) currX, currY = target[0], target[1] } return int(cost) } </pre>
priorityqueue .go	<pre> package model type Item struct { State *State Priority int // UCS: Cost, A*: Cost + heuristic </pre>

	<pre> index int } type PriorityQueue []*Item func (pq PriorityQueue) Len() int { return len(pq) } func (pq PriorityQueue) Less(i, j int) bool { return pq[i].Priority < pq[j].Priority // MIN } func (pq PriorityQueue) Swap(i, j int) { pq[i], pq[j] = pq[j], pq[i] pq[i].index = i pq[j].index = j } func (pq *PriorityQueue) Push(x any) { item := x.(*Item) item.index = len(*pq) *pq = append(*pq, item) } func (pq *PriorityQueue) Pop() any { old := *pq n := len(old) item := old[n-1] old[n-1] = nil *pq = old[:n-1] return item } </pre>
solver.go	<pre> package model import ("strings") type Solver struct { </pre>

```

Board      [][]byte
Costs      [][]int
GoalX, GoalY int
FinalNumber int
}

func (s Solver) isSolved(st State) bool {
    return st.X == s.GoalX && st.Y == s.GoalY && st.NextNumber ==
s.FinalNumber+1
}

func (s Solver) generateNextmoves(st State) []State {
    moves := make([]State, 0, 4)
    for _, d := range []rune{'U', 'D', 'L', 'R'} {
        state, canmove := s.Move(d, st)
        if canmove {
            moves = append(moves, state)
        }
    }
    return moves
}

func (s Solver) Move(dir rune, st State) (State, bool) {
    x, y := st.X, st.Y
    nextNumber := st.NextNumber
    cost := st.Cost

    for {
        nx, ny := x, y
        switch dir {
            case 'U': ny--
            case 'D': ny++
            case 'L': nx--
            case 'R': nx++
        }

        if ny < 0 || nx < 0 || ny >= len(s.Board) || nx >= len(s.Board[0])
|| s.Board[ny][nx] == 'L' {
            return State{}, false
        }
    }
}

```

```

        if s.Board[nx][ny] == 'X' {
            break
        }

        x, y = nx, ny
        cost += s.Costs[y][x]

        if s.Board[y][x] >= '0' && s.Board[y][x] <= '9' {
            currentNum := int(s.Board[y][x] - '0')
            if currentNum == nextNumber {
                nextNumber++
            } else if currentNum > nextNumber {
                return State{}, false
            }
        }
    }

    if st.X == x && st.Y == y {
        return State{}, false
    }

    return State{X: x, Y: y, NextNumber: nextNumber, Cost: cost, Parent:
&st}, true
}

func (s Solver) VisualizeState(st State) string {
    var sb strings.Builder
    for i := range len(s.Board) {
        for j := range len(s.Board[0]) {
            if j == st.X && i == st.Y {
                sb.WriteByte('Z')
                continue
            }

            tile := s.Board[i][j]
            if tile >= '0' && tile <= '9' {
                num := int(tile - '0')
                if num < st.NextNumber {
                    sb.WriteByte('*')
                    continue
                }
            }
        }
    }
}

```

```

    }

    sb.WriteByte(tile)
}
sb.WriteByte('\n')
}
return sb.String()
}

func (s Solver) GetRoute(st State) ([]*State, []rune) {
    states := []*State{}
    moves := []rune{}

    current := &st
    for current != nil {
        states = append(states, current)
        if current.Parent != nil {
            dx := current.X - current.Parent.X
            dy := current.Y - current.Parent.Y

            var dir rune
            if dx > 0 {
                dir = 'R'
            } else if dx < 0 {
                dir = 'L'
            } else if dy < 0 {
                dir = 'U'
            } else if dy > 0 {
                dir = 'D'
            } else {
                dir = '?'
            }

            moves = append(moves, dir)
        }
        current = current.Parent
    }

    for i, j := 0, len(states)-1; i < j; i, j = i+1, j-1 {
        states[i], states[j] = states[j], states[i]
    }

    for i, j := 0, len(moves)-1; i < j; i, j = i+1, j-1 {

```

	<pre> moves[i], moves[j] = moves[j], moves[i] } return states, moves } </pre>
state.go	<pre> package model // import "fmt" type State struct { X, Y int NextNumber int Cost int Parent *State } type StateKey struct { X, Y int NextNumber int } func (s State) Key() StateKey { return StateKey{X: s.X, Y: s.Y, NextNumber: s.NextNumber} } </pre>
app.js	<pre> document.addEventListener('DOMContentLoaded', () => { const form = document.getElementById('solveForm'); const algorithmSelect = document.getElementById('algorithm'); const heuristicGroup = document.getElementById('heuristicGroup'); const solveBtn = document.getElementById('solveBtn'); const btnText = document.getElementById('btnText'); const resultPanel = document.getElementById('resultPanel'); const downloadBtn = document.getElementById('downloadBtn'); // Playback elements const emptyStateMsg = document.getElementById('emptyStateMsg'); const boardContainer = document.getElementById('boardContainer'); const boardGrid = document.getElementById('boardGrid'); const actor = document.getElementById('actor'); const playbackControls = document.getElementById('playbackControls'); const stepSlider = document.getElementById('stepSlider'); </pre>

```

const stepCounter = document.getElementById('stepCounter');
const btnPlay = document.getElementById('btnPlay');
const iconPlay = document.getElementById('iconPlay');
const iconPause = document.getElementById('iconPause');
const btnPrev = document.getElementById('btnPrev');
const btnNext = document.getElementById('btnNext');

let currentResponse = null;
let currentStep = 0;
let isPlaying = false;
let playInterval = null;
let boardWidth = 0;
const TILE_SIZE = 40; // 40px width/height
const GAP_SIZE = 4; // gap-1 in tailwind = 4px

// Toggle heuristic dropdown
algorithmSelect.addEventListener('change', (e) => {
  const val = e.target.value;
  if (val === 'gbfs' || val === 'a*') {
    heuristicGroup.classList.remove('hidden');
  } else {
    heuristicGroup.classList.add('hidden');
  }
});

form.addEventListener('submit', async (e) => {
  e.preventDefault();

  const fileInput = document.getElementById('fileInput');
  if (!fileInput.files.length) return;

  const formData = new FormData();
  formData.append('file', fileInput.files[0]);
  formData.append('algorithm', algorithmSelect.value);
  formData.append('heuristic',
document.getElementById('heuristic').value);

  // Reset UI
  btnText.textContent = '⌚ Solving...';
  solveBtn.disabled = true;
  resultPanel.classList.add('hidden');

```



```

        playbackControls.classList.add('hidden');
        emptyStateMsg.classList.remove('hidden');
        boardContainer.classList.add('hidden');
        stepCounter.classList.add('hidden');
        pausePlayback();

        try {
            const res = await fetch('/api/solve', {
                method: 'POST',
                body: formData
            });
            const data = await res.json();

            if (!data.success) {
                alert(data.message);
                return;
            }

            currentResponse = data;

            // Fill result panel
            document.getElementById('resPath').textContent = data.path;
            document.getElementById('resCost').textContent =
data.states[data.states.length-1].Cost;
            document.getElementById('resTime').textContent = data.time_ms +
' ms';
            document.getElementById('resIter').textContent =
data.iterations;
            resultPanel.classList.remove('hidden');

            // Setup Board
            setupBoard(data);

            // Setup Playback
            currentStep = 0;
            stepSlider.max = data.states.length - 1;
            stepSlider.value = 0;
            playbackControls.classList.remove('hidden');
            emptyStateMsg.classList.add('hidden');
            boardContainer.classList.remove('hidden');
            stepCounter.classList.remove('hidden');

```

```

        updateBoardState(0);

    } catch (err) {
        alert('Terjadi kesalahan koneksi ke server.');
```

console.error(err);

```
    } finally {
        btnText.textContent = 'Solve Puzzle';
        solveBtn.disabled = false;
    }
});

function setupBoard(data) {
    boardWidth = data.board_width;
    const boardHeight = data.board_height;

    boardGrid.style.gridTemplateColumns = `repeat(${boardWidth},
    ${TILE_SIZE}px)`;
    boardGrid.style.gridTemplateRows = `repeat(${boardHeight},
    ${TILE_SIZE}px)`;
    boardGrid.innerHTML = '';

    for (let y = 0; y < boardHeight; y++) {
        for (let x = 0; x < boardWidth; x++) {
            const char = data.board[y][x];
            const tile = document.createElement('div');
            tile.className = 'board-tile text-sm ';

            if (char === 'X') {
                tile.classList.add('bg-slate-800', 'text-slate-800');
```

// Solid wall

```
            } else if (char === 'L') {
                tile.classList.add('bg-orange-500', 'text-white');
                tile.textContent = 'L';
            } else if (char === '*') {
                tile.classList.add('bg-slate-200');
            } else if (char >= '0' && char <= '9') {
                tile.classList.add('bg-emerald-500', 'text-white');
                tile.textContent = char;
                // Save id to easily dim it later
                tile.id = `tile-${x}-${y}`;
```

```

        } else if (char === 'O') {
            tile.classList.add('bg-emerald-700', 'text-white',
'rounded-full');

            tile.textContent = 'O';
        } else if (char === 'Z') {
            tile.classList.add('bg-slate-200'); // Initial Z spot
becomes empty
        } else {
            tile.classList.add('bg-slate-200');
        }

        boardGrid.appendChild(tile);
    }
}

function updateBoardState(stepIdx) {
    if (!currentResponse) return;
    const state = currentResponse.states[stepIdx];

    // Update actor position
    const posX = state.X * (TILE_SIZE + GAP_SIZE);
    const posY = state.Y * (TILE_SIZE + GAP_SIZE);
    actor.style.transform = `translate(${posX}px, ${posY}px)`;

    // Reset all tiles to default, then dim collected numbers
    const boardHeight = currentResponse.board_height;
    for (let y = 0; y < boardHeight; y++) {
        for (let x = 0; x < boardWidth; x++) {
            const char = currentResponse.board[y][x];
            if (char >= '0' && char <= '9') {
                const t = document.getElementById(`tile-${x}-${y}`);
                if (t) {
                    if (parseInt(char) < state.NextNumber) {
                        t.classList.replace('bg-emerald-500',
'bg-slate-300');

                        t.classList.replace('text-white',
'text-slate-400');
                    } else {
                        t.classList.replace('bg-slate-300',
'bg-emerald-500');

```

```

t.classList.replace('text-slate-400',
'text-white');

        }
    }
}

    }

    }

    stepSlider.value = stepIdx;
    stepCounter.textContent = `Step  ${stepIdx}  /
${currentResponse.states.length - 1}`;
    }

function playNext() {
    if (currentStep < currentResponse.states.length - 1) {
        currentStep++;
        updateBoardState(currentStep);
    } else {
        pausePlayback();
    }
}

function playPrev() {
    if (currentStep > 0) {
        currentStep--;
        updateBoardState(currentStep);
    }
}

function togglePlayback() {
    if (isPlaying) {
        pausePlayback();
    } else {
        if (currentStep >= currentResponse.states.length - 1) {
            currentStep = 0;
            updateBoardState(0);
        }
        isPlaying = true;
        if (iconPlay && iconPause) {
            iconPlay.classList.add('hidden');
            iconPause.classList.remove('hidden');
        }
    }
}

```

```

    }
    // Wait a tiny bit if resetting from end before sliding
    setTimeout(() => {
        if(!isPlaying) return;
        playInterval = setInterval(playNext, 400);
    }, 50);
}

function pausePlayback() {
    isPlaying = false;
    if (iconPlay && iconPause) {
        iconPlay.classList.remove('hidden');
        iconPause.classList.add('hidden');
    }
    if (playInterval) clearInterval(playInterval);
}

// Event Listeners for Playback
btnPlay.addEventListener('click', togglePlayback);
btnNext.addEventListener('click', () => { pausePlayback(); playNext();
});
btnPrev.addEventListener('click', () => { pausePlayback(); playPrev();
});
stepSlider.addEventListener('input', (e) => {
    pausePlayback();
    currentStep = parseInt(e.target.value);
    updateBoardState(currentStep);
});

// Download functionality
downloadBtn.addEventListener('click', () => {
    if (!currentResponse) return;

    let content = `[SOLUSI ICE SLIDING PUZZLE]\n`;
    content += `Algoritma:
${algorithmSelect.options[algorithmSelect.selectedIndex].text}\n`;
    if(algorithmSelect.value === 'gbfs' || algorithmSelect.value ===
'a*') {
        const h = document.getElementById('heuristic');
        content += `Heuristik: ${h.options[h.selectedIndex].text}\n`;
    }
}

```

```

    }
    content += `Rute: ${currentResponse.path}\n`;
                                content      +=      `Cost      Total:
${currentResponse.states[currentResponse.states.length-1].Cost}\n`;
    content += `Waktu Eksekusi: ${currentResponse.time_ms} ms\n`;
    content += `Banyak Iterasi: ${currentResponse.iterations}\n\n`;

    content += `[LANGKAH-LANGKAH]\n`;
    for (let i = 0; i < currentResponse.states.length; i++) {
        const state = currentResponse.states[i];
        const move = i === 0 ? "START" : currentResponse.path[i-1];
        content += `Step ${i} : ${move}\n`;

        // Draw board string
        for (let y = 0; y < currentResponse.board_height; y++) {
            let rowStr = "";
            for (let x = 0; x < currentResponse.board_width; x++) {
                if (x === state.X && y === state.Y) {
                    rowStr += "Z";
                    continue;
                }
                const char = currentResponse.board[y][x];
                if (char >= '0' && char <= '9' && parseInt(char) <
state.NextNumber) {
                    rowStr += "*";
                } else if (char === 'Z') {
                    rowStr += "*";
                } else {
                    rowStr += char;
                }
            }
            content += rowStr + "\n";
        }
        content += "\n";
    }

    const blob = new Blob([content], { type: "text/plain;charset=utf-8"
});

    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;

```

	<pre> const fn = document.getElementById('fileInput').files[0].name.split('.')[0]; a.download = `solusi_\${fn}_\${algorithmSelect.value}.txt`; document.body.appendChild(a); a.click(); document.body.removeChild(a); URL.revokeObjectURL(url); }); }); </pre>
index.html	<pre> <!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>Ice Sliding Puzzle Solver</title> <script src="https://cdn.tailwindcss.com"></script> <style> .board-tile { width: 40px; height: 40px; display: flex; align-items: center; justify-content: center; font-weight: bold; border-radius: 4px; } .actor-tile { transition: transform 0.25s cubic-bezier(0.4, 0, 0.2, 1); position: absolute; top: 0; left: 0; width: 40px; height: 40px; display: flex; align-items: center; justify-content: center; font-weight: bold; border-radius: 4px; z-index: 10; } </style> </pre>

```

</style>
</head>
<body class="bg-slate-50 text-slate-900 min-h-screen flex flex-col font-sans">
  <header class="bg-white border-b border-slate-200 px-6 py-4 shadow-sm flex justify-between items-center">
    <h1 class="text-2xl font-bold text-slate-800 flex items-center gap-3">
      <svg class="w-8 h-8 text-blue-500" fill="none" stroke="currentColor" viewBox="0 0 24 24" xmlns="http://www.w3.org/2000/svg"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M19 11H5m14 0a2 2 0 012 2v6a2 2 0 01-2 2H5a2 2 0 01-2 2v-6a2 2 0 012 2M5 11V9a2 2 0 00-2 2M5 11V9a2 2 0 012 2m0 0V5a2 2 0 012 2h6a2 2 0 012 2v2M7 7h10"></path></svg>
      Ice Sliding Puzzle Solver
    </h1>
  </header>

  <main class="flex-1 max-w-7xl w-full mx-auto p-6 grid grid-cols-1 lg:grid-cols-12 gap-8">
    <!-- Panel Kiri (Kontrol) -->
    <section class="lg:col-span-4 space-y-6">
      <div class="bg-white p-6 rounded-xl shadow-sm border border-slate-200">
        <form id="solveForm" class="space-y-5">
          <div>
            <label class="block text-sm font-medium mb-2">Upload Map (.txt)</label>
            <input type="file" id="fileInput" accept=".txt" required

            class="block w-full text-sm text-slate-500
              file:mr-4 file:py-2 file:px-4
              file:rounded-lg file:border-0
              file:text-sm file:font-semibold
              file:bg-blue-50 file:text-blue-700
              hover:file:bg-blue-100 cursor-pointer
            border border-slate-200 rounded-lg p-2" />
          </div>

          <div>
            <label class="block text-sm font-medium

```



```

mb-2">Algoritma</label>
                                <select id="algorithm" class="w-full border
border-slate-200      rounded-lg      p-2.5      bg-slate-50      focus:ring-2
focus:ring-blue-500 outline-none">
                                <option value="ucs">Uniform Cost Search
(UCS)</option>
                                <option value="gbfs">Greedy Best-First Search
(GBFS)</option>
                                <option value="a*">A* (A-Star)</option>
                                <option value="bfs">Breadth-First Search
(BFS)</option>
                                </select>
                            </div>

                            <div id="heuristicGroup" class="hidden">
                                <label class="block text-sm font-medium
mb-2">Heuristik</label>
                                <select id="heuristic" class="w-full border
border-slate-200      rounded-lg      p-2.5      bg-slate-50      focus:ring-2
focus:ring-blue-500 outline-none">
                                    <option value="h1">H1: Manhattan
Distance</option>
                                    <option value="h2">H2: Euclidean
Distance</option>
                                    <option value="h3">H3: Chebyshev
Distance</option>
                                    <option value="h4">H4: Missing Targets</option>
                                    <option value="h5">H5: Straight to
Goal</option>
                                </select>
                            </div>

                            <button type="submit" id="solveBtn"
                                class="w-full bg-blue-600 hover:bg-blue-700
text-white font-semibold py-3 px-4 rounded-lg transition-colors flex
justify-center items-center gap-2">
                                <span id="btnText">Solve Puzzle</span>
                            </button>
                        </form>
                    </div>

```

	<pre> <!-- Panel Hasil --> <div id="resultPanel" class="bg-white p-6 rounded-xl shadow-sm border border-slate-200 hidden"> <h3 class="font-bold text-lg mb-4 border-b pb-2">Metrik Penyelesaian</h3> <div class="space-y-3 text-sm"> <div class="flex justify-between">Rute: </div> <div class="flex justify-between">Cost Total: </div> <div class="flex justify-between">Waktu: </div> <div class="flex justify-between">Iterasi: </div> </div> <button id="downloadBtn" class="mt-5 w-full bg-slate-800 hover:bg-slate-900 text-white py-2.5 rounded-lg text-sm font-medium transition-colors flex justify-center items-center gap-2"> <svg class="w-5 h-5" fill="none" stroke="currentColor" viewBox="0 0 24 24" xmlns="http://www.w3.org/2000/svg"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M4 16v1a3 3 0 03 3h10a3 3 0 03-3v-1m-4-4l-4 4m0 0l-4-4m4 4V4"></path></svg> Download Log Solusi (.txt) </button> </div> </section> <!-- Panel Kanan (Visualisasi Map) --> <section class="lg:col-span-8 flex flex-col"> <div class="bg-white flex-1 rounded-xl shadow-sm border border-slate-200 p-6 flex flex-col min-h-[500px]"> <div class="flex justify-between items-center mb-4"> <h2 class="text-lg font-bold">Visualisasi Papan</h2> Step 0 / 0 </div> </pre>
--	---

	<pre> <!-- Container Map --> <div class="flex-1 bg-slate-50 rounded-lg border border-slate-200 overflow-auto flex items-center justify-center p-4 relative"> <p id="emptyStateMsg" class="text-slate-400">Papan permainan akan muncul di sini</p> <div id="boardContainer" class="relative hidden"> <!-- Grid Background --> <div id="boardGrid" class="grid gap-1"></div> <!-- Aktor Animasi --> <div id="actor" class="actor-tile bg-blue-500 text-white shadow-md border border-blue-600">Z</div> </div> </div> <!-- Kontrol Playback --> <div id="playbackControls" class="mt-6 flex flex-col gap-4 hidden"> <input type="range" id="stepSlider" min="0" max="0" value="0" class="w-full accent-blue-600 cursor-pointer"> <div class="flex justify-center items-center gap-4"> <button id="btnPrev" class="p-2 rounded-full hover:bg-slate-100 text-slate-600 transition-colors" title="Mundur"> <svg class="w-6 h-6" fill="currentColor" viewBox="0 0 20 20" xmlns="http://www.w3.org/2000/svg"><path fill-rule="evenodd" d="M15.707 15.707a1 1 0 01-1.414 01-5-5a1 1 0 010-1.41415-5a1 1 0 111.414 1.414L11.414 1014.293 4.293a1 1 0 010 1.414zm-6 0a1 1 0 01-1.414 01-5-5a1 1 0 010-1.41415-5a1 1 0 011.414 1.414L5.414 1014.293 4.293a1 1 0 010 1.414z" clip-rule="evenodd"></path></svg> </button> <button id="btnPlay" class="p-3 bg-blue-100 hover:bg-blue-200 text-blue-700 rounded-full transition-colors font-bold w-12 h-12 flex items-center justify-center" title="Play/Pause"> <svg id="iconPlay" class="w-6 h-6" fill="currentColor" viewBox="0 0 20 20" xmlns="http://www.w3.org/2000/svg"><path fill-rule="evenodd" d="M10 18a8 8 0 100-16 8 8 0 000 16zM9.555 7.168A1 1 0 008 8v4a1 1 0 001.555.83213-2a1 1 0 000-1.6641-3-2z" clip-rule="evenodd"></path></svg> <svg id="iconPause" class="w-6 h-6 hidden" fill="currentColor" viewBox="0 0 20 20" </pre>
--	---


```

BoardWidth int      `json:"board_width"`
BoardHeight int     `json:"board_height"`
Board      []string  `json:"board"`
Costs      [][]int   `json:"costs"`
}

func getTargets(solver model.Solver) map[int][2]int {
    targets := make(map[int][2]int)
    for i := 0; i < len(solver.Board); i++ {
        for j := 0; j < len(solver.Board[0]); j++ {
            c := solver.Board[i][j]
            if c >= '0' && c <= '9' {
                targets[int(c-'0')] = [2]int{j, i}
            } else if c == 'O' {
                targets[solver.FinalNumber+1] = [2]int{j, i}
            }
        }
    }
    return targets
}

func solveHandler(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodPost {
        http.Error(w, "Method not allowed",
http.StatusMethodNotAllowed)
        return
    }

    err := r.ParseMultipartForm(10 << 20)
    if err != nil {
        http.Error(w, err.Error(), http.StatusBadRequest)
        return
    }

    file, _, err := r.FormFile("file")
    if err != nil {
        http.Error(w, "Error retrieving the file",
http.StatusBadRequest)
        return
    }
    defer file.Close()

```

```

algorithm := r.FormValue("algorithm")
heuristicName := r.FormValue("heuristic")

defer func() {
    if r := recover(); r != nil {
        w.Header().Set("Content-Type", "application/json")
        json.NewEncoder(w).Encode(SolveResponse{Success: false,
Message: fmt.Sprintf("Format file tidak valid: %v", r)})
    }
}()

initialState, solver := utils.ParseReader(file)

var hFunc model.HeuristicFunc
targets := getTargets(solver)
if algorithm == "gbfs" || algorithm == "a*" {
    switch heuristicName {
    case "h1":
        hFunc = model.H1Manhattan(targets)
    case "h2":
        hFunc = model.H2Euclidean(targets)
    case "h3":
        hFunc = model.H3Chebyshev(targets)
    case "h4":
        hFunc = model.H4MissingTargets()
    case "h5":
        hFunc = model.H5StraightToGoal(targets)
    default:
        hFunc = model.H1Manhattan(targets)
    }
}

start := time.Now()
var result model.AlgorithmResult

switch algorithm {
case "ucs":
    result = solver.UCS(initialState)
case "gbfs":
    result = solver.GBFS(initialState, hFunc)

```

```

        case "a*":
            result = solver.AStar(initialState, hFunc)
        case "bfs":
            result = solver.BFS(initialState)
        default:
            result = solver.UCS(initialState)
    }

    elapsed := time.Since(start).Milliseconds()

    if !result.Solved {
        w.Header().Set("Content-Type", "application/json")
        json.NewEncoder(w).Encode(SolveResponse{Success: false,
Message: "Solusi tidak ditemukan!"})
        return
    }

    states, pathRunes := solver.GetRoute(result.FinalState)
    path := string(pathRunes)
    iterations := result.Iterations

    boardStr := make([]string, len(solver.Board))
    for i, row := range solver.Board {
        boardStr[i] = string(row)
    }

    resp := SolveResponse{
        Success:    true,
        Path:        path,
        TimeMs:      elapsed,
        Iterations:  iterations,
        States:      states,
        BoardWidth:  len(solver.Board[0]),
        BoardHeight: len(solver.Board),
        Board:       boardStr,
        Costs:       solver.Costs,
    }

    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(resp)
}

```

```
func main() {  
    http.Handle("/", http.FileServer(http.Dir("./static")))  
    http.HandleFunc("/api/solve", solveHandler)  
  
    fmt.Println("Server Web berjalan di http://localhost:8080")  
    fmt.Println("Tekan Ctrl+C untuk berhenti.")  
    if err := http.ListenAndServe(":8080", nil); err != nil {  
        fmt.Println("Gagal menjalankan server:", err)  
    }  
}
```


V. Hasil Program

Percobaan 1	
Input	5 6 XXXXXX XZ*1*X X*X*OX X0**XX XXXXXX 999 999 999 999 999 999 999 1 1 2 1 999 999 1 999 1 1 999 999 2 1 1 1 999 999 999 999 999 999 999
Output	Algoritma: Uniform Cost Search (UCS) Rute: DURD Cost Total: 10 Waktu Eksekusi: 0 ms Banyak Iterasi: 11 [LANGKAH-LANGKAH] Step 0 : START XXXXXX XZ*1*X X*X*OX X0**XX XXXXXX Step 1 : D XXXXXX

	X**1*X X*X*OX XZ**XX XXXXXX Step 2 : U XXXXXX XZ*1*X X*X*OX X***XX XXXXXX Step 3 : R XXXXXX X***ZX X*X*OX X***XX XXXXXX Step 4 : D XXXXXX X*****X X*X*ZX X***XX XXXXXX
Analisis	Pada pengujian ini, algoritma <i>Uniform Cost Search (UCS)</i> terbukti berhasil menemukan jalur penyelesaian yang dijamin optimal secara mutlak dengan <i>total cost</i> pergerakan sebesar 10. Keterbatasan percabangan (<i>branching factor</i>) akibat banyaknya rintangan pada ukuran peta yang kecil mempermudah pencarian. Hal ini terlihat jelas dari performa pencarian di mana <i>UCS</i> hanya

	memerlukan 11 iterasi ekspansi <i>state</i> dan waktu eksekusi yang nyaris instan (0 ms). Keberadaan mekanisme <i>Closed List</i> pada <i>UCS</i> juga terbukti bekerja sangat baik karena berhasil mencegah program terjebak dalam <i>infinite loop</i> saat aktor harus melakukan gerakan bolak-balik (<i>Down-Up</i>) pada lorong vertikal yang sama.
--	--

Percobaan 2	
Input	7 8 XXXXXXXXX XZ*****X X*XXXX*X X0*L**1X X*XXXX*X X***OX*X XXXXXXXXX 999 999 999 999 999 999 999 999 999 1 1 1 1 1 1 999 999 1 999 999 999 999 1 999 999 2 1 999 1 2 1 999 999 1 999 999 999 999 1 999 999 1 1 1 1 999 1 999 999 999 999 999 999 999 999 999
Output	Algoritma: Greedy Best-First Search (GBFS) Heuristik: H2: Euclidean Distance Rute: DURDULDR Cost Total: 36 Waktu Eksekusi: 0 ms Banyak Iterasi: 11

[LANGKAH-LANGKAH]

Step 0 : START

XXXXXXXXXX

XZ*****X

X*XXXX*X

X0*L**1X

X*XXXX*X

X***OX*X

XXXXXXXXXX

Step 1 : D

XXXXXXXXXX

X*****X

X*XXXX*X

X**L**1X

X*XXXX*X

XZ**OX*X

XXXXXXXXXX

Step 2 : U

XXXXXXXXXX

XZ*****X

X*XXXX*X

X**L**1X

X*XXXX*X

X***OX*X

XXXXXXXXXX

Step 3 : R

XXXXXXXXXX

X*****ZX
X*XXXX*X
X**L**IX
X*XXXX*X
X***OX*X
XXXXXXXXX

Step 4 : D
XXXXXXXXX
X*****X
X*XXXX*X
X**L***X
X*XXXX*X
X***OXZX
XXXXXXXXX

Step 5 : U
XXXXXXXXX
X*****ZX
X*XXXX*X
X**L***X
X*XXXX*X
X***OX*X
XXXXXXXXX

Step 6 : L
XXXXXXXXX
XZ*****X
X*XXXX*X
X**L***X

	<pre> X*XXXX*X X***OX*X XXXXXXXXX Step 7 : D XXXXXXXXX X*****X X*XXXX*X X**L***X X*XXXX*X XZ**OX*X XXXXXXXXX Step 8 : R XXXXXXXXX X*****X X*XXXX*X X**L***X X*XXXX*X X***ZX*X XXXXXXXXX </pre>
Analisis	Pada pengujian ini, algoritma <i>Greedy Best-First Search (GBFS)</i> dengan heuristik <i>Euclidean Distance</i> menunjukkan efisiensi yang luar biasa. Meskipun peta memiliki jebakan <i>Lava</i> dan menuntut rute memutar yang cukup panjang, <i>GBFS</i> berhasil menemukan solusi dalam waktu nyaris instan (0 ms) dengan hanya 11 iterasi ekspansi <i>state</i>. Hal ini membuktikan bahwa nilai heuristik mampu memandu arah pencarian dengan sangat presisi, sehingga algoritma langsung memilih rute yang tepat tanpa melakukan percabangan (<i>branching</i>) yang sia-sia. Walaupun secara teoritis <i>GBFS</i> tidak menjamin solusi dengan <i>cost</i> terkecil, sifat pencariannya yang sangat agresif terbukti efektif dalam

	bermanuver melewati rintangan.
--	--------------------------------

Percobaan 3	
Input	8 8 XXXXXXXXX XZ*X***X X*X**X*X X**X*X*X X0**X**X X*XX*X*X X1*2**OX XXXXXXXXX 999 999 999 999 999 999 999 999 999 1 1 999 2 2 1 999 999 1 999 1 2 999 1 999 999 1 1 999 2 999 1 999 999 2 1 1 999 2 2 999 999 1 999 999 1 999 1 999 999 1 1 2 1 1 1 999 999 999 999 999 999 999 999 999
Output	Algoritma: A* (A-Star) Heuristik: H3: Chebyshev Distance Rute: DR Cost Total: 12 Waktu Eksekusi: 0 ms Banyak Iterasi: 4 [LANGKAH-LANGKAH] Step 0 : START

	XXXXXXXXXX XZ*X***X X*X**X*X X**X*X*X X0**X**X X*XX*X*X X1*2**OX XXXXXXXXXX Step 1 : D XXXXXXXXXX X**X***X X*X**X*X X**X*X*X X***X**X X*XX*X*X XZ*2**OX XXXXXXXXXX Step 2 : R XXXXXXXXXX X**X***X X*X**X*X X**X*X*X X***X**X X*XX*X*X X*****ZX XXXXXXXXXX
Analisis	Pada pengujian ini, algoritma A^* (<i>A-Star</i>) dengan heuristik <i>Chebyshev Distance</i> menunjukkan keseimbangan performa yang sangat optimal. Peta uji

	ini secara unik memiliki susunan angka target yang sejajar lurus secara vertikal maupun horizontal. Algoritma A^* secara brilian berhasil memanfaatkan celah tersebut untuk meraup seluruh <i>checkpoint</i> (0, 1, dan 2) hanya melalui dua luncuran panjang beruntun ($D-R$). Perpaduan antara perhitungan <i>cost</i> aktual dan panduan heuristik membuat algoritma ini langsung terkunci pada rute berbiaya minimum (12) tanpa sedikitpun salah arah. Hal ini dibuktikan dari waktu eksekusi 0 ms dan jumlah iterasi yang sangat kecil, yakni hanya 4 ekspansi <i>state</i>.
--	--

Percobaan 4	
Input	6 10 XXXXXXXXXXXX XZ*****0*X X**XX***1X X*X***X*XX X*XO***L*X XXXXXXXXXXXX 999 999 999 999 999 999 999 999 999 999 1 1 999 1 1 1 1 999 999 2 2 3 1 999 2 2 2 999 999 1 1 1 1 5 1 999 1 999 999 999 1 999 1 1 1 1 999 999 999 999 999 999 999 999 999 999 999
Output	Algoritma: A^* (A-Star) Heuristik: H4: Missing Targets Rute: RDLDL Cost Total: 3016 Waktu Eksekusi: 0 ms Banyak Iterasi: 15

[LANGKAH-LANGKAH]

Step 0 : START

XXXXXXXXXXXX

XZ*****0*X

X**XX***1X

X*X***X*XX

X*XO***L*X

XXXXXXXXXXXX

Step 1 : R

XXXXXXXXXXXX

X*****ZX

X**XX***1X

X*X***X*XX

X*XO***L*X

XXXXXXXXXXXX

Step 2 : D

XXXXXXXXXXXX

X*****X

X**XX***ZX

X*X***X*XX

X*XO***L*X

XXXXXXXXXXXX

Step 3 : L

XXXXXXXXXXXX

X*****X

X**XXZ***X

	<pre> X*X**X*XX X*XO***L*X XXXXXXXXXX Step 4 : D XXXXXXXXXX X*****X X**XX***X X*X**X*XX X*XO*Z*L*X XXXXXXXXXX Step 5 : L XXXXXXXXXX X*****X X**XX***X X*X**X*XX X*XZ***L*X XXXXXXXXXX </pre>
Analisis	Pada pengujian terakhir ini, algoritma A^* dengan heuristik <i>Missing Targets</i> dihadapkan pada peta berkarakteristik area terbuka yang sangat luas. Minimnya rintangan di area tengah memaksa aktor untuk meluncur jauh ke dinding terluar demi bisa berbelok arah. Manuver ini mengharuskan aktor menginjak ubin dengan matriks <i>cost</i> penalti yang ekstrem, sehingga menghasilkan <i>total biaya</i> yang sangat besar (3016). Meskipun biayanya membengkak, A^* menjamin bahwa rute <i>zig-zag</i> ($R-D-L-D-L$) tersebut adalah rute paling optimal yang secara absolut bisa dilalui. Heuristik <i>Missing Targets</i> juga terbukti efektif memandu penyelesaian <i>puzzle</i> yang sangat mengecoh ini hanya dalam 15 iterasi dan waktu 0 ms tanpa membiarkan aktor tersesat di ruang terbuka.

Percobaan 5	
Input	7 7 XXXXXXXX X0****X X**X**X X****OX X1***LX XZ**X*X XXXXXXXX 999 999 999 999 999 999 999 999 3 5 2 8 1 999 999 7 4 999 6 9 999 999 2 8 3 5 4 999 999 6 1 7 2 999 999 999 9 3 4 999 8 999 999 999 999 999 999 999 999
Output	Algoritma: Breadth-First Search (BFS) Rute: RULUDRUR Cost Total: 87 Waktu Eksekusi: 0 ms Banyak Iterasi: 16 [LANGKAH-LANGKAH] Step 0 : START XXXXXXXX X0****X X**X**X X****OX

X1***LX
XZ**X*X
XXXXXXX

Step 1 : R
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
X**ZX*X
XXXXXXX

Step 2 : U
XXXXXXX
X0****X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXX

Step 3 : L
XXXXXXX
X0****X
X**X**X
XZ***OX
X1***LX
X***X*X
XXXXXXX

Step 4 : U

XXXXXXXX

XZ****X

X**X**X

X****OX

X1***LX

X***X*X

XXXXXXXX

Step 5 : D

XXXXXXXX

X*****X

X**X**X

X****OX

X****LX

XZ**X*X

XXXXXXXX

Step 6 : R

XXXXXXXX

X*****X

X**X**X

X****OX

X****LX

X**ZX*X

XXXXXXXX

Step 7 : U

XXXXXXXX

	<pre> X*****X X**X**X X**Z*OX X*****LX X***X*X XXXXXXXX Step 8 : R XXXXXXXX X*****X X**X**X X****ZX X*****LX X***X*X XXXXXXXX </pre>
Analisis	Pada pengujian ini, algoritma <i>Breadth-First Search (BFS)</i> digunakan untuk menyelesaikan peta yang memiliki tingkat kerumitan menengah dengan adanya jebakan <i>Lava (L)</i>. Sebagai algoritma pencarian buta (<i>uninformed search</i>), <i>BFS</i> mengeksplorasi <i>state</i> secara <i>level-by-level</i> murni berdasarkan kedalaman gerakan, tanpa memperhitungkan sama sekali bobot matriks <i>cost</i>. Meskipun arah pencariannya menyebar rata secara sistematis (terlihat dari angka ekspansi sebanyak 16 iterasi), program tetap mampu menghindari jebakan <i>Lava</i> dan menemukan rute memutar sepanjang 8 langkah (<i>R-U-L-U-D-R-U-R</i>) dalam waktu instan (0 ms). Sesuai dengan teori, solusi yang dihasilkan <i>BFS</i> dijamin merupakan rute terpendek secara "jumlah langkah pergerakan", namun total biaya rutenya (87) tidak dijamin sebagai lintasan dengan <i>cost</i> pergerakan paling minimum.

VI. *Link Repository*

Kode program lengkap beserta instruksi penggunaan dan *file executable* dapat diakses melalui *link repository* berikut:

https://github.com/wisauce/Tucil3_18223003_18223015

VII. **Surat Pernyataan**

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Wisma Ahmaduta Dinutama



Muhammad Adam Mirza

VIII. Lampiran

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan		
2	Program berhasil dijalankan		
3	Solusi yang diberikan program benar dan mematuhi aturan permainan		
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt		
5	Program memiliki Graphical User Interface (GUI)		
6	Ada algoritma pathfinding alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)		
7	Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)		